

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-a51f984285e157fb6.jsonl`
- SHA-256: `ada084a443d5653173c17a20e007500789ea543f910c48fbaea69c24e23ec9ea`
- Source modified: `2026-06-22T01:28:56+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_2be29ecf-a31`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:26:03.396Z)

You are reviewing PR #310 "P2b-install" for CROSS-PLATFORM CORRECTNESS of scripts/install.py + scripts/uninstall.py.

Run `cd C:\Users\avidu\Projects\bhi-p2b && git diff origin/master...HEAD` and read scripts/install.py + scripts/uninstall.py in C:\Users\avidu\Projects\bhi-p2b. Scrutinize: (1) shortcut_spec ? are the Windows .bat (CRLF, set "RALLY_APP_HOME=..."), macOS .command, Linux .sh contents correct + runnable? Does the path embedding survive Windows backslashes? (2) install_command/uninstall_command argv for uv vs pip correct? (3) resolve_method + default_os_app_home per-OS logic mirror backend.utils.app_home correctly? (4) Does the launcher shortcut's RALLY_APP_HOME actually take effect given load_dotenv runs at engine import (the stated load-bearing claim)? (5) Any failure mode: uv present but unusable, pip into wrong env, shortcut not on PATH, missing dirs? Report concrete findings only with fixes. Empty list if nothing real.

2. assistant (2026-06-22T01:26:06.271Z)

I'll review PR #310 for cross-platform correctness. Let me start by examining the diff and the key files.

3. user (2026-06-22T01:26:09.309Z)

```
78e423d docs(pkg): link P2b-install PR #310 in the §7 tracker
a5c1493 feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run default (gate 08)
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers (#309)
de2bf14 feat(pkg): P2e ? lazy-torch so the LIGHT tier imports with NO torch installed (default-OFF) (#308)
679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export) (default-OFF) (#307)
```

4. assistant (2026-06-22T01:26:12.084Z)

Let me look at the diff for the two files in question and read them in full.

5. user (2026-06-22T01:26:13.143Z)

README.md		31	++++++
backend/config/__init__.py		9	+-
backend/config/loader.py		34	++++++
backend/main.py		15	++++
docs/PACKAGING_PLAN.md		13	+-
scripts/install.py		210	+++++
scripts/uninstall.py		150	+++++

```
tests/test_app_launcher.py | 42 ++++++++
tests/test_p2b_install.py | 163 ++++++
9 files changed, 660 insertions(+), 7 deletions(-)
```

6. user (2026-06-22T01:26:18.374Z)

```
1      #!/usr/bin/env python3
2      """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
P2b).
3
4      Cross-platform (Windows / macOS / Linux), no native component (01):
5
6      * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7      env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8      * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9      ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10     (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11     starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12     launcher (not by the engine) roots the ``.env`` in the app-home;
13     * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15     The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16     ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17     this script only wires the install + the launcher.
18
19     Usage::
20
21     python scripts/install.py                # install '.' (this checkout), uv-
if-present else pip
22     python scripts/install.py --target badminton-highlight-indexer  # from PyPI (once
published)
23     python scripts/install.py --method pip    # force pip
24     python scripts/install.py --dry-run       # print the plan, change nothing
25
26     stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
27     """
28     from __future__ import annotations
29
30     import argparse
31     import json
32     import os
33     import platform
34     import shutil
35     import subprocess
36     import sys
37     from datetime import datetime, timezone
38     from pathlib import Path
39
40     PACKAGE = "badminton-highlight-indexer"
41     APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
42     MANIFEST_NAME = "uninstall_manifest.json"
43
44
45     # --- pure helpers (unit-tested)
-----
46
47     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
48                             env: dict[str, str] | None = None) -> Path:
49         """The OS-conventional per-user app-data dir ? a standalone mirror of
50         ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
```

```

exists)."""
51     system = system or platform.system()
52     e = os.environ if env is None else env
53     if system == "Windows":
54         base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
55     elif system == "Darwin":
56         base = str(Path.home() / "Library" / "Application Support")
57     else:
58         base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
59     return Path(base) / app_name
60
61
62     def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
tuple[str, str, bool]:
63         """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.
64
65         The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so
it's testable.
66         """
67         home = str(app_home)
68         if system == "Windows":
69             content = (
70                 "@echo off\r\n"
71                 "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
72                 f'set "RALLY_APP_HOME={home}"\r\n'
73                 f"{indexer_cmd} --app\r\n"
74             )
75             return "KhelSutra.bat", content, False
76         if system == "Darwin":
77             content = (
78                 "#!/bin/bash\n"
79                 "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
80                 f'export RALLY_APP_HOME="{home}"\n'
81                 f"exec {indexer_cmd} --app\n"
82             )
83             return "KhelSutra.command", content, True
84         # Linux / other
85         content = (
86             "#!/bin/bash\n"
87             "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
88             f'export RALLY_APP_HOME="{home}"\n'
89             f"exec {indexer_cmd} --app\n"
90         )
91         return "khelsutra.sh", content, True
92
93
94     def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
95                       timestamp: str) -> dict:
96         """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to
remove);
97         ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
98         return {
99             "schema": 1,
100             "product": "KhelSutra highlight indexer",
101             "package": PACKAGE,
102             "install_method": method,          # "uv" | "pip"
103             "indexer_command": "indexer",
104             "app_home": str(app_home),
105             "shortcut": str(shortcut),
106             "created": [str(p) for p in created],
107             "user_data": [
108                 str(app_home / "config.json"),

```

```

109         str(app_home / ".env"),
110         str(app_home / "output"),
111     ],
112     "timestamp": timestamp,
113 }
114
115
116 def resolve_method(requested: str, uv_present: bool) -> str:
117     """Pick the installer: explicit beats auto; auto prefers uv when present, else
118     pip."""
119     if requested in ("uv", "pip"):
120         return requested
121     return "uv" if uv_present else "pip"
122
123 def install_command(method: str, target: str) -> list[str]:
124     """The argv for the chosen installer (PURE)."""
125     if method == "uv":
126         return ["uv", "tool", "install", target]
127     return [sys.executable, "-m", "pip", "install", target]
128
129
130 # --- IO / orchestration
131 -----
132
133 def _run(argv: list[str]) -> None:
134     print(f" $ {' '.join(argv)}")
135     subprocess.run(argv, check=True)
136
137
138 def main(argv: list[str] | None = None) -> int:
139     ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer
140     (LIGHT tier).")
141     ap.add_argument("--target", default=".",
142                     help="What to install: '.' (this checkout, default) or a PyPI
143     name/spec.")
144     ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
145                     help="Installer to use (default: uv if present, else pip).")
146     ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
147     nothing.")
148     args = ap.parse_args(argv)
149
150     system = platform.system()
151     app_home = default_os_app_home()
152     method = resolve_method(args.method, shutil.which("uv") is not None)
153     cmd = install_command(method, args.target)
154     fname, content, executable = shortcut_spec(system, app_home)
155     shortcut = app_home / fname
156
157     print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
158     print(f" OS : {system}")
159     print(f" installer : {method}")
160     print(f" target : {args.target}")
161     print(f" app-home : {app_home}")
162     print(f" shortcut : {shortcut}")
163     if args.dry_run:
164         print("\n[dry-run] would run:")
165         print(f" $ {' '.join(cmd)}")
166         print(f" write shortcut -> {shortcut}")
167         print(f" write manifest -> {app_home / MANIFEST_NAME}")
168         print(f" copy uninstaller -> {app_home / 'uninstall.py'}")
169         return 0
170
171     print("\n[1/3] installing the package?")
172     try:

```

```

169         _run(cmd)
170     except (subprocess.CalledProcessError, FileNotFoundError) as e:
171         print(f"[ERROR] install failed: {e}")
172         if method == "uv":
173             print(" (uv not usable? re-run with --method pip)")
174         return 1
175
176     print("\n[2/3] writing the launcher shortcut + app-home?")
177     app_home.mkdir(parents=True, exist_ok=True)
178     shortcut.write_text(content, encoding="utf-8")
179     if executable:
180         shortcut.chmod(0o755)
181     # Make the uninstaller available next to the data it removes.
182     created: list[Path] = [shortcut]
183     here = Path(__file__).resolve().parent
184     src_uninstall = here / "uninstall.py"
185     dst_uninstall = app_home / "uninstall.py"
186     if src_uninstall.exists():
187         shutil.copyfile(src_uninstall, dst_uninstall)
188         created.append(dst_uninstall)
189
190     print("\n[3/3] writing the uninstall manifest?")
191     manifest_path = app_home / MANIFEST_NAME
192     created.append(manifest_path)
193     manifest = build_manifest(
194         method=method, app_home=app_home, shortcut=shortcut, created=created,
195         timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
196     )
197     manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
198
199     print("\n? Installed. To launch KhelSutra:")
200     if system == "Windows":
201         print(f"    double-click {shortcut} (or run `indexer --app`)")
202     else:
203         print(f"    {shortcut} (or run `indexer --app`)")
204     print("    The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
205     print(f"    To uninstall: python \"{dst_uninstall}\" (add --purge to also delete
your data)")
206     return 0
207
208
209 if __name__ == "__main__":
210     raise SystemExit(main())
211

```

7. user (2026-06-22T01:26:18.549Z)

```

1     #!/usr/bin/env python3
2     """Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).
3
4     Reverses ``scripts/install.py`` from the manifest it wrote:
5
6     * uninstalls the package (`uv tool uninstall` / `pip uninstall -y` ? matches the
install method);
7     * removes the files the installer created (launcher shortcut, the copied uninstaller,
the manifest);
8     * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the
Gemini key,
9     and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.
10
11     Usage::
12
13     python uninstall.py                # find the manifest in the app-home, keep user
data

```

```

14         python uninstall.py --purge             # also delete config.json / .env / output (your
data)
15         python uninstall.py --manifest /path/to/uninstall_manifest.json
16         python uninstall.py --dry-run           # print the plan, change nothing
17
18     stdlib-only; safe to run from the copy the installer dropped in the app-home.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import json
24     import os
25     import platform
26     import shutil
27     import subprocess
28     import sys
29     from pathlib import Path
30
31     APP_DIR_NAME = "Khelsutra"
32     MANIFEST_NAME = "uninstall_manifest.json"
33
34
35     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
36                             env: dict[str, str] | None = None) -> Path:
37         """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not
38         given)."""
39         system = system or platform.system()
40         e = os.environ if env is None else env
41         if system == "Windows":
42             base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
43         elif system == "Darwin":
44             base = str(Path.home() / "Library" / "Application Support")
45         else:
46             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
47         return Path(base) / app_name
48
49     def find_manifest(explicit: str | None) -> Path | None:
50         """The manifest path: explicit arg, else next to THIS file, else the default app-
51         home."""
52         if explicit:
53             p = Path(explicit)
54             return p if p.exists() else None
55         for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
56                     default_os_app_home() / MANIFEST_NAME):
57             if cand.exists():
58                 return cand
59         return None
60
61     def uninstall_command(method: str, package: str) -> list[str]:
62         """The argv to remove the package (PURE)."""
63         if method == "uv":
64             return ["uv", "tool", "uninstall", package]
65         return [sys.executable, "-m", "pip", "uninstall", "-y", package]
66
67
68     def _rm(path: Path, dry: bool) -> None:
69         try:
70             if path.is_dir() and not path.is_symlink():
71                 print(f" - rmtree {path}")
72                 if not dry:
73                     shutil.rmtree(path, ignore_errors=True)
74             elif path.exists() or path.is_symlink():
75                 print(f" - remove {path}")

```

```

76         if not dry:
77             path.unlink(missing_ok=True)
78     except OSError as e:
79         print(f" ! could not remove {path}: {e}")
80
81
82     def main(argv: list[str] | None = None) -> int:
83         ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight
indexer.")
84         ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
85         ap.add_argument("--purge", action="store_true",
86                         help="Also delete YOUR data (config.json, .env, output/). Default
keeps it.")
87         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
88         args = ap.parse_args(argv)
89
90         manifest_path = find_manifest(args.manifest)
91         if not manifest_path:
92             print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
93             return 1
94         manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
95         method = manifest.get("install_method", "pip")
96         package = manifest.get("package", "badminton-highlight-indexer")
97         created = [Path(p) for p in manifest.get("created", [])]
98         user_data = [Path(p) for p in manifest.get("user_data", [])]
99         app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
100
101         print(f"KhelSutra uninstaller ? {package} (installed via {method})")
102         print(f" manifest : {manifest_path}")
103         print(f" app-home : {app_home}")
104         print(f" data      : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use
--purge to delete)'}")
105         dry = args.dry_run
106
107         print("\n[1/3] removing the package?")
108         cmd = uninstall_command(method, package)
109         print(f" $ {' '.join(cmd)}")
110         if not dry:
111             try:
112                 subprocess.run(cmd, check=False)
113             except FileNotFoundError as e:
114                 print(f" ! {method} not found ({e}); remove the package manually.")
115
116         print("\n[2/3] removing installer-created files?")
117         # Remove the manifest LAST (we still reference it); skip the running uninstaller
copy itself.
118         self_path = Path(__file__).resolve()
119         for p in created:
120             if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
121                 continue
122             _rm(p, dry)
123
124         if args.purge:
125             print("\n[3/3] purging user data (--purge)?")
126             for p in user_data:
127                 _rm(p, dry)
128         else:
129             print("\n[3/3] keeping your data:")
130             for p in user_data:
131                 print(f"   · {p}")
132
133         # Manifest + (optionally) the empty app-home last.
134         _rm(manifest_path, dry)
135         if args.purge and app_home.exists():

```

```

136         try:
137             remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
138             if not remaining:
139                 _rm(app_home, dry)
140         except OSError:
141             pass
142
143         print("\n? Uninstalled." + (" " if args.purge else " Your data was kept. "))
144         if self_path.parent == app_home:
145             print(f"    (you can delete this script + the app-home folder manually:
{app_home})")
146         return 0
147
148
149     if __name__ == "__main__":
150         raise SystemExit(main())
151

```

8. assistant (2026-06-22T01:26:21.353Z)

Now let me read the actual engine's `backend.utils.app\_home` to verify the mirroring claim, and the `load\_dotenv` / config loader to verify the load-bearing claim.

9. user (2026-06-22T01:26:23.542Z)

```
./backend/utils/app_home.py
```

```
---loader---
```

```
"""Configuration loader and saver.
```

Provides a single place to load validated AppConfig with built-in defaults merged with optional file overrides (config.json).

This replaces scattered json.load + .get() patterns throughout the codebase.

```
from __future__ import annotations
```

```
import json
```

```
import logging
```

```
import os
```

```
from pathlib import Path
```

```
from typing import Any
```

```
from pydantic import BaseModel
```

```
from backend.utils.app_home import default_config_path
```

```
from .models import AppConfig
```

```
from .presets import (
```

```
    COMPUTE_TARGET_FOR_PROFILE,
```

```
    PROFILE_PRESETS,
```

```
    is_known_profile,
```

```
    preset_for,
```

```
    suggest_profile,
```

```
)
```

```
logger = logging.getLogger(__name__)
```

Config path resolution now lives in ``backend.utils.app\_home.default\_config\_path()``
``RALLY\_APP\_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")``
the env is unset, exactly the pre-P0a value. (The old module-level ``DEFAULT\_CONFIG``
was dropped: it had no external consumers and a frozen-at-import snapshot would sil

from a later RALLY_APP_HOME.)

```
def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
    """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge, scalars
    replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
    overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-profile
    path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
    out = dict(base)
    for key, val in override.items():
        if isinstance(val, dict) and isinstance(out.get(key), dict):
            out[key] = _deep_merge(out[key], val)
        else:
            out[key] = val
    return out

def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
    """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if none
    is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so a bare
    environment never silently selects a profile (and never silently spends on cloud).

    An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the file's
    profile (rather than suppressing it) ? so an env typo can't leave the recorded profile
    asserting a preset that was never applied. A bad value in config.json is left to surface as
    a hard, typed-Literal error downstream (config-file correctness)."""
    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
    if env_profile and env_profile.strip():
        ep = env_profile.strip()
        if is_known_profile(ep):
            return ep
        logger.warning(
            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring it "
            "and falling back to config.json deployment.profile.", ep, "
        )
    ".join(PROFILE_PRESETS),
    )
    # fall through to the file's profile (env typo must not suppress a valid file choice)
    dep = file_data.get("deployment")
    if isinstance(dep, dict):
        prof = dep.get("profile")
        if isinstance(prof, str) and prof.strip():
            return prof.strip()
    return ""

def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str = "") ->
list[str]:
```

10. user (2026-06-22T01:26:26.143Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.
2
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
6      from
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
8      silently
9      "loses" their index. This module introduces ONE resolver every state path roots on.
10     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
11     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
    sets it).
12     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
```

```

behaviour.
12
13     The OS-default location (`%APPDATA%` / `~/Library/Application Support` /
`~/local/share`)
14     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15     calls it and exports `RALLY_APP_HOME` explicitly. Keeping the resolver opt-in means
importing
16     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17     real user profile during a test run).
18
19     stdlib-only (os, platform, pathlib) so it is safe to import at `backend.main` line 1 ?
before
20     the heavier pydantic-backed `backend.config` package ? keeping the early
`load_dotenv` path light.
21     """
22
23     from __future__ import annotations
24
25     import os
26     import platform
27     from pathlib import Path
28
29     #: Environment variable the packaged launcher sets to pin the app-data home.
30     APP_HOME_ENV = "RALLY_APP_HOME"
31
32     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33     APP_DIR_NAME = "Khelsutra"
34
35
36     def app_home() -> Path:
37         """The resolved application-data home.
38
39         `RALLY_APP_HOME` (`~` expanded) when set; otherwise `Path(".")` ? the CWD,
byte-identical
40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. `app_path("output")`)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """`<app-home>/config.json` ? the config file the loader reads and `--setup`
writes.
55
56         With `RALLY_APP_HOME` unset this is `Path("config.json")` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """`<app-home>/.env` ? the dotenv file the server prefers (e.g.
`GEMINI_API_KEY`)."""
63         return app_home() / ".env"
64
65

```

```

66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv()
search
76             return None
77         candidate = home / ".env"
78         return candidate if candidate.exists() else None
79
80
81     def resolve_output_dir(output_dir: str) -> str:
82         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
83
84         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
85         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
86         normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
87         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
88         app's state; an absolute path ? or a Windows drive-anchored one (``C:\x`` /
``C:x``) ? passes
89         through untouched.
90         """
91         home = app_home()
92         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
93             return output_dir or "output"
94         if not output_dir:
95             return str(home / "output")
96         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
97             return output_dir
98         return str(home / output_dir)
99
100
101     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
102         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
103         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
104
105         * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
106         * macOS ? ``~/Library/Application Support/<app_name>``
107         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
108         """
109         system = platform.system()
110         if system == "Windows":
111             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
112         elif system == "Darwin":
113             base = str(Path.home() / "Library" / "Application Support")
114         else:
115             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
116         return Path(base) / app_name
117

```

11. assistant (2026-06-22T01:26:29.730Z)

The `default\_os\_app\_home` mirror matches exactly. Now let me verify the load-bearing `load\_dotenv` claim by checking `backend/main.py` and how `env\_file\_to\_load` is consumed at import time.

12. user (2026-06-22T01:26:30.591Z)

```
1     import os
2     import sys
3     import argparse
4     import json
5     import logging
6     import shutil
7     import socket
8     import threading
9     import uuid
10    import webbrowser
11    from importlib.metadata import PackageNotFoundError, version as _pkg_version
12    from urllib.parse import quote
13    import uvicorn
14    from dotenv import load_dotenv
15
16    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
17    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
18    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
19    # stdlib-only import ? keeps this pre-pydantic line light.
20    from backend.utils.app_home import env_file_to_load, resolve_output_dir
21
22    load_dotenv(env_file_to_load())
23
24    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
25    logging.basicConfig(
26        level=logging.INFO,
27        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
28        datefmt='%H:%M:%S'
29    )
30    logger = logging.getLogger(__name__)
31    from fastapi import FastAPI, HTTPException, Request
32    from fastapi.middleware.cors import CORSMiddleware
33    from starlette.middleware.trustedhost import TrustedHostMiddleware
34    from pydantic import BaseModel
35    from typing import List, Dict, Any, Optional
36
37    from fastapi.staticfiles import StaticFiles
38    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
39    from backend.pipeline.validators.base import registry
40    from backend.infrastructure.database import Database
41    from backend.pipeline.segmenters.base import segmenter_registry
42    from backend.pipeline.stitcher.compiler import VideoCompiler
43    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
44    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
45    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
46    from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
47    from backend.utils.single_instance import acquire_lock # P0e: one instance per database
48    from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
49    from backend.config import ( # new typed config
50        load_config as load_app_config,
```

```

51     write_light_default_config, # P2b: batteries-included first-run config seeding
52     AppConfig,
53     StitchingConfig,
54     enforce_startup_checks,
55     StartupCheckError,
56 )
57 from backend.results import Failure # standardized error/result contract
58 from backend.reporting import emit_indexer_report
59 from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
60 from backend import billing # M8: per-job cost ledger (USD internal / INR display)
61 from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
62
63 app = FastAPI(title="Sports Highlight Indexer API")
64
65
66 def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
67     """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
68     default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
69     does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
70     src = os.environ if env is None else env

```

13. assistant (2026-06-22T01:26:35.070Z)

The load-bearing claim is verified: ``load_dotenv(env_file_to_load())`` runs at line 22 of ``backend/main.py``, at import time. ``env_file_to_load()`` returns the app-home ``env`` only when ``RALLY_APP_HOME`` is set. So the launcher must set the env BEFORE the process starts ? which the shortcut does. The claim is correct.

Now let me check the ``--app`` flag handling and how ``indexer`` entrypoint resolves, plus verify the Windows .bat path-embedding behavior with backslashes. Let me look at the test file and the entry point.

14. user (2026-06-22T01:26:35.511Z)

```

1     """P2b-install (PACKAGING_PLAN.md): batteries-included config seeding +
install/uninstall tooling.
2
3     Covers:
4     * the engine-side ``write_light_default_config`` (gate 08 default = truly-local +
motion), incl.
5     the load-bearing **no-overwrite** idempotency and the EFFECTIVE config the loader
produces;
6     * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut
content, manifest,
7     installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/I/O.
8     """
9     from __future__ import annotations
10
11     import importlib.util
12     import json
13     from pathlib import Path
14
15     import pytest
16
17     from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
18
19
20     def _load_script(name: str):
21         """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to
exec)."""

```

```

22     path = Path(__file__).resolve().parents[1] / "scripts" / name
23     spec = importlib.util.spec_from_file_location(name[:-3], path)
24     assert spec and spec.loader
25     mod = importlib.util.module_from_spec(spec)
26     spec.loader.exec_module(mod)
27     return mod
28
29
30     # --- engine seeding: write_light_default_config (gate 08)
-----
31
32     def test_seed_writes_truly_local_motion(tmp_path):
33         cfg = tmp_path / "sub" / "config.json" # also proves parent-dir creation
34         path, written = write_light_default_config(cfg)
35         assert written is True and path == cfg and cfg.exists()
36         data = json.loads(cfg.read_text(encoding="utf-8"))
37         assert data == LIGHT_DEFAULT_CONFIG
38         assert data["indexing"]["default_segmenter"] == "motion"
39         assert data["indexing"]["use_gpu"] is False
40         assert data["deployment"]["profile"] == "truly-local"
41
42
43     def test_seed_does_not_overwrite_existing(tmp_path):
44         cfg = tmp_path / "config.json"
45         cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}',
encoding="utf-8")
46         path, written = write_light_default_config(cfg)
47         assert written is False and path == cfg
48         # The user's file is preserved untouched (the wizard/edits are never clobbered).
49         assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"]
== "gemini"
50
51
52     def test_seed_overwrite_true_forces(tmp_path):
53         cfg = tmp_path / "config.json"
54         cfg.write_text("{}", encoding="utf-8")
55         _, written = write_light_default_config(cfg, overwrite=True)
56         assert written is True
57         assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG
58
59
60     def test_seeded_config_loads_to_truly_local_motion(tmp_path):
61         """The minimal seeded file, run through the loader, expands via the truly-local
PRESET to a
62         coherent torch-free/offline config (motion + no Gemini handoff + local storage + no
GPU)."""
63         cfg = tmp_path / "config.json"
64         write_light_default_config(cfg)
65         loaded = load_config(cfg)
66         assert loaded.indexing.default_segmenter == "motion"
67         assert loaded.indexing.use_gpu is False
68         assert loaded.deployment.profile == "truly-local"
69         assert loaded.indexing.skip_ai_handoff is True # supplied by the truly-local
preset
70         assert loaded.storage.backend == "local" # supplied by the truly-local
preset
71
72
73     # --- scripts/install.py pure logic
-----
74
75     @pytest.fixture(scope="module")
76     def install_mod():
77         return _load_script("install.py")
78

```

```

79
80 @pytest.fixture(scope="module")
81 def uninstall_mod():
82     return _load_script("uninstall.py")
83
84
85 @pytest.mark.parametrize(
86     "system, fname, executable",
87     [("Windows", "KhelSutra.bat", False), ("Darwin", "KhelSutra.command", True),
88      ("Linux", "khelsutra.sh", True)],
89 )
90 def test_shortcut_spec_per_os(install_mod, system, fname, executable):
91     home = Path("/home/u/AppHome")
92     name, content, ex = install_mod.shortcut_spec(system, home)
93     assert name == fname and ex is executable
94     # str(home) is OS-native (backslashes on windows) ? assert against it, not a
hardcoded literal.
95     assert "RALLY_APP_HOME" in content and str(home) in content
96     assert "indexer --app" in content
97
98
99 def test_resolve_method(install_mod):
100     assert install_mod.resolve_method("auto", uv_present=True) == "uv"
101     assert install_mod.resolve_method("auto", uv_present=False) == "pip"
102     assert install_mod.resolve_method("pip", uv_present=True) == "pip" # explicit wins
103     assert install_mod.resolve_method("uv", uv_present=False) == "uv"
104
105
106 def test_install_command_argv(install_mod):
107     assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]
108     pip = install_mod.install_command("pip", "badminton-highlight-indexer")
109     assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
110
111
112 def test_default_os_app_home_per_os(install_mod):
113     win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
r"C:\Users\u\AppData\Roaming"})
114     assert win.name == "Khelsutra" and "Roaming" in str(win)
115     lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
"/home/u/.local/share"})
116     assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
117
118
119 def test_build_manifest_shape(install_mod):
120     home = Path("/home/u/AppHome")
121     m = install_mod.build_manifest(
122         method="uv", app_home=home, shortcut=home / "khelsutra.sh",
123         created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
timestamp="2026-06-22T00:00:00+00:00",
124     )
125     assert m["package"] == "badminton-highlight-indexer"
126     assert m["install_method"] == "uv"
127     assert str(home / "config.json") in m["user_data"] # user data is tracked
separately (kept on uninstall)
128     assert m["shortcut"] == str(home / "khelsutra.sh")
129     assert m["schema"] == 1
130
131
132 # --- scripts/uninstall.py pure logic
-----
133
134 def test_uninstall_command_argv(uninstall_mod):
135     assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
136         ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
137     pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")

```

```

138         assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]
139
140
141     def test_find_manifest_explicit(uninstall_mod, tmp_path):
142         m = tmp_path / "uninstall_manifest.json"
143         m.write_text("{} ", encoding="utf-8")
144         assert uninstall_mod.find_manifest(str(m)) == m
145         assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None
146
147
148     def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
149         """The manifest install.py would write is consumable by uninstall.py's
150         expectations."""
151         home = tmp_path / "AppHome"
152         m = install_mod.build_manifest(
153             method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
154             created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
155             timestamp="2026-06-22T00:00:00+00:00",
156         )
157         mp = home / "uninstall_manifest.json"
158         home.mkdir(parents=True)
159         mp.write_text(json.dumps(m), encoding="utf-8")
160         assert uninstall_mod.find_manifest(str(mp)) == mp
161         loaded = json.loads(mp.read_text(encoding="utf-8"))
162         assert uninstall_mod.uninstall_command(loaded["install_method"],
163             loaded["package"])[0:3] in (
164             ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
165         )
166

```

15. assistant (2026-06-22T01:26:39.706Z)

Now let me verify how the `indexer` console entrypoint is defined and whether `uv tool install` would actually put it on PATH, and check `--app`. Let me check pyproject/setup for the entrypoint.

16. user (2026-06-22T01:26:41.300Z)

```

pyproject.toml:84:[project.scripts]
pyproject.toml:86:indexer = "backend.main:main"

```

17. assistant (2026-06-22T01:26:50.009Z)

The `indexer` entrypoint maps to `backend.main:main`. Now let me check the key cross-platform concern: the Windows `.bat` `set "RALLY\_APP\_HOME=.."` syntax with backslashes, and whether `indexer --app` resolves in the `.bat` context (PATH after `uv tool install` vs pip). Let me also check whether the `.bat`'s `set "VAR=value"` handles trailing backslash issues and whether `indexer` is found. Let me verify a subtle issue: the `.bat` uses `set "RALLY\_APP\_HOME=C:\Users\...\Roaming\Khelsutra"` ? a path ending in a backslash inside quotes would be fine, but a path NOT ending in backslash is fine too. Let me confirm there's no trailing-backslash issue and check the `uv tool install` PATH behavior.

18. user (2026-06-22T01:26:53.555Z)

```

FILENAME: KhelSutra.bat
--- CONTENT (repr to see CRLF + backslashes) ---
'@echo off\r\nrem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\r\nset "RALLY_APP_HOME=C:\\Users\\u\\AppData\\Roaming\\Khelsutra"\r\nindexer --app\r\n'
--- rendered ---
@echo off

rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the app.

set "RALLY_APP_HOME=C:\Users\u\AppData\Roaming\Khelsutra"

```


indexer --app

19. assistant (2026-06-22T01:26:58.384Z)

The Windows `.bat` content is correct: CRLF line endings, `set "RALLY_APP_HOME=..."` with literal backslashes (cmd.exe does not treat backslash as an escape, so the path survives intact), no trailing backslash. The path embedding survives Windows backslashes correctly.

Now I notice an important encoding issue. The `.bat` content contains a non-ASCII em-dash (```) in the `rem` comment, and the file is written with `encoding="utf-8"`. Let me investigate whether that causes problems for cmd.exe, and check the `@echo off` UTF-8 BOM concern.

20. user (2026-06-22T01:27:01.240Z)

First bytes (BOM check): `b'@ec'`
Has em-dash byte seq (`e2 80 94`): `True`
Total bytes: 171

21. assistant (2026-06-22T01:27:07.989Z)

No BOM (good ? a UTF-8 BOM would break `@echo off`). The em-dash is only in a `rem` comment line, so even if cmd.exe misrenders it under a non-UTF-8 codepage, it's inert (a comment). Not a real failure.

Now let me examine the more substantive cross-platform failure modes. Let me check: (1) on macOS/Linux, the `.command/.sh` shortcut is written into the app-data dir but is NOT on PATH and double-clicking a `.sh` on Linux typically doesn't execute ? but the script's output message accounts for that. (2) The critical PATH issue: after `pip install` (not `uv tool install`), is `indexer` actually on PATH for the `.bat/.command`? Let me check the macOS/Linux `--app` desktop integration claim, and whether `uv tool install .` works with a relative `.` target.

22. user (2026-06-22T01:27:09.259Z)

```
11:app.add_middleware(
21:    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
33:    # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
40:app.add_middleware(
63:# --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? 01)
---
83:def _resolve_app_port(host: str, port: int) -> int:
84:    """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
113:# Serves static frontend files directly (now under api/ per approved structure).
124:app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
```

23. user (2026-06-22T01:27:13.762Z)

```
132:# --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? 01)
---
153:    """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
861:def main():
863:    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
864:    parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
865:    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
866:    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video
clip to run detailed diagnostics on")
867:    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
868:    parser.add_argument("--app", action="store_true",
871:    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
```

```

server on")
872:     parser.add_argument("--host", default=None,
875:     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")
878:     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
879:     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
880:     parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug
trim segment feature")
881:     parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")
938:     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly `--app`
942:     # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER overwrites
an
943:     # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to
--app
945:     if args.app:
996:     # CLI processing mode (no web UI needed). --app/--server always run the server, never
CLI single-shot.
997:     if args.video_path and not args.server and not args.app:
1078:     if args.app or args.server or not args.video_path:
1079:         if args.app:
1089:             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).

```

24. assistant (2026-06-22T01:27:19.578Z)

`indexer --app` is a real, supported flag. The entrypoint and behavior are consistent with the launcher.

Now let me look at the most significant cross-platform correctness concern I've identified: the `uv tool install` PATH problem. `uv tool` puts binaries in `~/.local/bin` (Unix) or `%APPDATA%\uv\tools` / `%LOCALAPPDATA%\bin` shim dir on Windows, which may NOT be on PATH unless `uv tool update-shell`/`uv tool ensurepath` was run. Similarly `pip install` into a user/system env may put `indexer` in a Scripts dir not on PATH. The launcher invokes bare `indexer --app` ? if it's not on PATH, the shortcut fails. Let me verify whether the script does anything to ensure PATH, and check the `.command`/`.sh` double-click execution behavior.

25. user (2026-06-22T01:27:21.082Z)

```

scripts/install.py:7:     env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
exit: 0

```

26. assistant (2026-06-22T01:27:24.575Z)

Confirmed ? no PATH-ensuring step. Now let me verify the `uv tool install` behavior with a relative target, and whether `uv tool install` from a local checkout`.` is even supported (vs `--from`). Let me also check the README for the documented install flow and whether the wheel is published.

27. user (2026-06-22T01:27:25.422Z)

```

diff --git a/README.md b/README.md
index 674e304..0bd46dd 100644
--- a/README.md
+++ b/README.md
@@ -32,6 +32,37 @@ The pipeline is built around three pluggable registries ? **validators**,
**se

## ?? Setup & Diagnostics

#### 0. Batteries-included install (LIGHT tier ? recommended for end users)
+

```

```

+For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-platform
+installer. It installs the **LIGHT tier** (torch-free: Gemini / `motion` / cpu-mock), wires a
+desktop launcher, and seeds a sensible default config.
+
+```bash
+# Preferred ? uv puts `indexer` on your PATH in an isolated tool env:
+uv tool install badminton-highlight-indexer          # (once published) or:  uv tool install .
+
+# Or the cross-platform installer script (uv-if-present, else pip) ? also creates a launcher
+# shortcut + an uninstall manifest:
+python scripts/install.py                            # add --method pip to force pip, --dry-run
to preview
+```
+
+Then launch the app (the installer's shortcut, or directly):
+
+```bash
+indexer --app          # boots a local server (127.0.0.1) and opens the bundled KhelSutra UI
+```
+
+* **First run seeds a truly-local default** ? detector `motion` (offline, **no API key, no GPU,
+ no cloud bills**); the in-app setup later switches you to Gemini. State (config, DB, reels)
lives
+ in your OS app-data dir (`%APPDATA%\Khelsutra` / `~/Library/Application Support/Khelsutra` /
+ `~/.local/share/Khelsutra`) when launched via the installer's shortcut.
+* **FFmpeg is still required** (see step 1) ? it is intentionally not redistributed (GPL
`libx264`).
+* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your
config/reels
+ by default; add `--purge` to delete them too.
+
+The native-GPU (WASB) and cloud tiers are separate, opt-in provisioning ? see
`docs/PACKAGING_PLAN.md`.
+
+### 1. External System Requirements
+The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes,
and stitch clips.

```

28. assistant (2026-06-22T01:27:29.681Z)

The README explicitly claims "uv puts `indexer` on your PATH in an isolated tool env". This is only true if `uv tool ensurepath` / `uv tool update-shell` was run ? `uv tool install` itself prints a warning when its bin dir is not on PATH but does NOT add it. This is a real first-run failure mode for the launcher shortcut.

Let me now run the test suite for this PR's portion to confirm the pure logic is green, and check the macOS `.command` / Linux `.sh` double-click concern more concretely.

29. user (2026-06-22T01:27:42.495Z)

```

..... [100%]
22 passed in 9.80s

```

30. assistant (2026-06-22T01:27:52.478Z)

Tests pass. Now let me examine one more subtle uninstall correctness issue on Windows: `pip uninstall -y` when installed via `uv tool install` (or vice versa), and the self-deletion of the running uninstaller `.py` on Windows (you cannot delete a running script on Windows... actually you can delete a `.py` while the interpreter runs it, since Python reads it fully into memory ? but the manifest references and `self\_path` skip handle this). Let me verify the uninstaller's handling when run from the app-home copy, and the `app\_home` empty-check logic.

Let me look closely at the uninstall self-skip and the manifest `created` list. The installer adds `dst\_uninstall` to `created`, and the uninstaller skips `self\_path` and `manifest\_path`.

But the copied uninstaller in app-home is named `uninstall.py` while the installer copies from `scripts/uninstall.py`. When the user runs `python uninstall.py` from app-home, `self\_path` = app-home/uninstall.py, which IS in `created`, so it's correctly skipped and left behind (then the final message tells them to delete manually). Good.

Now let me check a real edge case: the `--purge` empty-app-home removal filters out `self\_path` but not the manifest (already removed) ? and on Windows the leftover `uninstall.py` (skipped) means app-home won't be empty, so it won't be removed. That's consistent with the message. Fine.

Let me verify the `uv tool install .` relative-path concern and the cross-method uninstall mismatch.

31. user (2026-06-22T01:27:53.671Z)

```
17:name = "badminton-highlight-indexer"
23:authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
---
```

MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storage

```
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]
```

```
[project.scripts]
```

```
`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
```

```
indexer = "backend.main:main"
```

```
[tool.hatch.metadata]
```

The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct reference*); hatchling requires opting in to allow those in metadata.

32. assistant (2026-06-22T01:28:10.389Z)

Now I understand the full picture. The package name `badminton-highlight-indexer` matches the manifest default, and the entrypoint is correct.

One more critical check: `uv tool install .` from a checkout containing a git-direct-reference dependency (the `reporting` extra pins `obsreport` to a git tag). But that's only an extra, not a default dep ? the default `uv tool install .` won't pull it. Let me verify the default deps are torch-free / installable without the git ref.

Let me also double check the `find\_manifest` ambiguity: when run from `scripts/` in the repo (not app-home), `find\_manifest` looks next to the file first ? but there's no manifest in `scripts/`, so it falls to default app-home. That's fine.

Let me verify the one remaining concern about `pip install .` putting `indexer` in a Scripts dir not on PATH, and whether `--target .` with pip vs uv could install into the wrong environment (e.g., if `python scripts/install.py` is run with a system python, `sys.executable -m pip install .` installs into that python's site-packages ? could be a system/global env without PATH-exposed scripts).

I have enough to report. Let me compile the findings. Let me do a final verification on the `set "VAR=path"` trailing-backslash edge case ? if APPDATA itself somehow had a trailing separator, the joined path with `Khelsutra` leaf never ends in backslash, so `set "RALLY\_APP\_HOME=...Khelsutra"` is always safe. Confirmed safe.

Let me verify the macOS/Linux exec line: `exec indexer --app` ? if `indexer` isn't on PATH, the `command`/`.sh` fails identically. And on Linux, a double-clicked `.sh` in most desktop environments either opens in an editor or prompts ? but the script's message says to run it, and it's chmod +x. This is a minor UX gap, not a correctness bug, since the message also gives `indexer --app`.

Now let me check one genuine bug candidate: the installer writes the shortcut and copies `uninstall.py`, but does the `command`/`.sh` shortcut actually get found/launched? It's placed

in the app-data dir, which is NOT a desktop or applications folder. The README and install output both tell the user the full path. So it's "create a launcher file" not "register a desktop entry". The docstring says "creates a per-OS launcher shortcut" and "double-click" ? but placing it in `~/.local/share/Khelsutra/khelsutra.sh` (Linux) or `~/Library/Application Support/Khelsutra/KhelSutra.command` (macOS) means it's buried, not on the desktop/Applications/Start Menu. This is a UX limitation but the messaging is honest about the path. I'll note it as minor.

Let me confirm there are no other consumers and finalize.

33. user (2026-06-22T01:28:13.239Z)

```
=== Darwin: KhelSutra.command (exec=True) ===
'#!/bin/bash\n# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\nexport RALLY_APP_HOME="\Users\u\Library\Application Support\Khelsutra"\nexec indexer
--app\n'
=== Linux: khelsutra.sh (exec=True) ===
'#!/bin/bash\n# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\nexport RALLY_APP_HOME="\Users\u\Library\Application Support\Khelsutra"\nexec indexer
--app\n'
```

34. assistant (2026-06-22T01:28:28.350Z)

That output is misleading because I'm running on Windows so `pathlib.Path` produced backslashes ? on a real macOS/Linux host, `Path` is `PosixPath` and `str()` yields forward slashes. So the rendered `command`/`.sh` would correctly be `export RALLY\_APP\_HOME="/Users/u/Library/Application Support/Khelsutra"`. The space in "Application Support" is properly double-quoted. That's fine.

But this reveals a genuine concern worth checking precisely: the path is embedded in double quotes in the shell script. A path containing a literal `\$`, backtick, or `` would break it ? but OS app-data paths under a username could contain such characters only if the username does (rare). The double-quoting handles spaces correctly, which is the common case (macOS "Application Support"). Not a real bug.

Let me verify the one thing I want to be certain about ? whether `str(PosixPath)` with the space is correctly quoted (it is, via the `f'export RALLY\_APP\_HOME="{home}"`). Confirmed in source.

I now have all findings. Let me compile the structured output.

The findings:

1. **Major** ? launcher's bare `indexer` relies on PATH that neither `uv tool install` nor `pip install` guarantees. The README claims "uv puts `indexer` on your PATH", but `uv tool install` does not modify PATH; it requires `uv tool update-shell`/`ensurepath`. With `pip install` into a system/global Python (which `sys.executable -m pip install .` targets), the Scripts dir is often not on PATH either. The shortcut runs bare `indexer --app`, so on a fresh machine the launcher can fail with "command not found".
2. **Minor** ? the "launcher shortcut" is written into the app-data dir, not a discoverable location (Desktop/Start Menu/Applications). Docstring/README say "double-click", but Linux desktops generally don't execute a `.sh` on double-click, and the file is buried. Messaging gives the full path + `indexer --app` fallback, so it's a UX gap not a hard break.
3. **Nit** ? `pip install` may target the wrong environment. `python scripts/install.py` resolves pip via `sys.executable`, so whichever Python ran the script gets the package. If launched with a system Python, `indexer` lands globally; combined with finding #1, the launcher may not find it.

Let me reconsider severity of #1 ? is it really a defect in *these two files* vs documentation? The shortcut content itself is correct; the gap is that nothing ensures PATH. It's a real cross-platform failure mode of the install flow, so major is appropriate but I'll frame the fix concretely.

35. user (2026-06-22T01:28:43.721Z)

Structured output provided successfully

36. assistant (2026-06-22T01:28:56.301Z)

Review complete. I scrutinized `scripts/install.py` and `scripts/uninstall.py` in `C:\Users\avidu\Projects\bhi-p2b` against the engine code.

What is correct (verified, no findings):

- **shortcut_spec contents** ? Windows `.bat` uses CRLF, `set "RALLY\_APP\_HOME=..."`, no BOM, backslash path survives intact (cmd.exe doesn't escape backslashes); the em-dash is only in an inert `rem` comment. macOS `.command` / Linux `.sh` use `#!/bin/bash`, `export RALLY\_APP\_HOME="..."` (double-quoted, so the "Application Support" space is safe), `exec indexer --app`. On a real POSIX host `str(PosixPath)` yields forward slashes correctly.
- **install/uninstall argv** ? `uv tool install/uninstall <target>` and `sys.executable -m pip install / pip uninstall -y <pkg>` are both correct; package name `badminton-highlight-indexer` matches pyproject.
- **resolve_method + default_os_app_home** ? byte-for-byte mirror of `backend/utils/app\_home.py::default\_os\_app\_home` (APPDATA / Library/Application Support / XDG_DATA_HOME, leaf `Khelsutra`).
- **The load-bearing RALLY_APP_HOME claim is real** ? `backend/main.py:22` calls `load\_dotenv(env\_file\_to\_load())` at import, and `env\_file\_to\_load()` only returns the app-home `.env` when `RALLY\_APP\_HOME` is set; the shortcut sets it before the process starts, so it takes effect. `indexer --app` is a supported flag (`backend/main.py:868`).
- Uninstall self-skip / manifest-last ordering / keep-data-by-default logic is sound. 22 pure-logic tests pass.

Two findings returned (1 major, 1 minor) via StructuredOutput:

- Major** ? the launcher runs bare `indexer`, but neither `uv tool install` nor `pip install` guarantees it's on PATH (README's "uv puts indexer on your PATH" is false without `uv tool update-shell`; `sys.executable -m pip` can install into a system env whose Scripts dir isn't on PATH). Fresh-machine shortcut can fail with command-not-found.
- Minor** ? the "launcher shortcut" lands in the app-data dir (not Desktop/Start Menu/Applications), and Linux double-click of a `.sh` usually doesn't execute; the "double-click" framing overstates it (though the path + `indexer --app` fallback is printed).